Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Report

on

"e-election"

[Code No: COMP 202]

For partial fulfillment of Second Year/first Semester in Computer Engineering

<u>Submitted By:</u>

Tosin Lamichhane [27]

<u>Submitted To:</u>

Mr. Sagar Acharya

Department of Computer Science and Engineering

Submission Date: 2026-02-25

Acknowledgement

I would like to express my sincere and heartfelt gratitude to my respected teacher, **Mr. Sagar Acharya**, for his continuous guidance, invaluable suggestions, and constant encouragement in the field of Data Structures and Algorithms. His excellent teaching, insightful explanations, and dedication to the subject inspired me to undertake and successfully complete this project. His support played a vital role in strengthening my understanding of the core concepts that formed the foundation of this work.

I would also like to extend my sincere appreciation to the Department of Computer Science and Engineering for providing me with the opportunity and platform to apply my theoretical knowledge in a practical and meaningful way. The academic environment, resources, and encouragement provided by the department were instrumental in helping me carry out and complete this project successfully.

Finally, I would like to express my gratitude to everyone who directly or indirectly supported and encouraged me during the course of this project. Their motivation and support contributed significantly to the successful completion of this work. I am truly thankful for their assistance and encouragement.

Thank You

Abstract

Data Structures and Algorithms are fundamental to the development of efficient and optimized software systems. This project presents the implementation of a **Nepal E-Election Vote Tallying System** using C++ with a graphical user interface developed using the Qt Framework. The primary objective of this project is to demonstrate the practical application of core data structures — namely Linked List, Queue, Stack, Binary Search Tree (BST), Hash Table, and Selection Sort — within an interactive and user-friendly electoral environment.

In this system, registered candidates are maintained using a **Singly Linked List**, which allows dynamic storage and sequential traversal of candidate records. The voter processing pipeline is implemented using a **Queue** (First-In-First-Out), ensuring that voters are handled in the order they arrive, accurately reflecting a real-world election queuing model. To support an undo mechanism, a **Stack** (Last-In-First-Out) is used to maintain a history of processed votes, enabling the reversal of the most recent vote when required. Candidate ID validation is performed using a **Binary Search Tree (BST)**, providing efficient search operations during vote processing. A **Hash Table** is employed to prevent duplicate voting by maintaining a registry of voter IDs that have already cast their votes. Finally, the live leaderboard is generated by applying **Selection Sort** on the candidate list, ranked in descending order of votes received.

The graphical user interface is developed using the Qt Framework, offering an interactive experience that includes candidate selection, voter ID entry, queue visualization, real-time leaderboard updates, and an activity log for monitoring system events.

Keywords: E-Election, Vote Tallying, Linked List, Queue, Stack, Binary Search Tree, Hash Table, Selection Sort, C++, Qt Framework, Data Structures, Graphical User Interface.

Table of Contents

List of Figures v

Acronyms/Abbreviations vi

Chapter 1 Introduction 1

1.1 Background 1

1.2 Objectives 2

1.3 Motivation and Significance 3

Chapter 2 Related Works 4

Chapter 3 Design and Implementation 5

3.1 Implementation 5

3.2 System Requirement Specifications 12

3.2.1 Software Specifications 12

3.2.2 Hardware Specifications 13

Chapter 4 Discussion on the achievements 14

4.1 Features 14

Chapter 5 Conclusion and Recommendation 18

5.1 Limitations 18

5.2 Future Enhancement 19

References 20

APPENDIX 21

List of Figures

Fig 3.1 System's Sequence Diagram..…….……..7

Fig A.1: System Interface……..…….…..20

Fig A.2: Game Completion Interface……..….20

Fig A.3: Stack Implementation Description Interface…….…21

Fig A.4: Queue Implementation Description Interface……21

Fig A.5: Tree Implementation Description Interface……22

Fig A.6: About Game Interface……22

Acronyms/Abbreviations

The list of all abbreviations used in the documentation are as follows :

DSA: Data Structures and Algorithms

GUI: Graphical User Interface

Qt: Qt Framework

LIFO: Last In, First Out

FIFO: First In, First Out

OOP: Object-Oriented Programming

IDE: Integrated Development Environment

API: Application Programming Interface

FPS: Frames Per Second

UI: User Interface

UX: User Experience

Chapter 1 Introduction

1.1 Background

Elections are one of the most important processes in any democracy, and making them fair, transparent, and efficient is always a challenge. This project came from a simple question, what if we could simulate how votes are actually collected and counted, using the data structures we study in class?

The Nepal E-Election Vote Tallying System is a desktop application built with C++ and Qt that lets users register voters, queue them up, process their votes, and watch a live leaderboard update in real time. Every feature in the system is backed by a data structure, not just for the sake of it, but because each one genuinely fits the problem it solves. A queue handles voters in order. A hash table stops someone from voting twice. A stack lets you undo a mistake. A BST validates candidates quickly.

Building this project made it clear that data structures are not just textbook concepts, they are practical tools that show up in real systems every single day. When you see a voter getting rejected because they already voted, that is a hash table doing its job. When the leaderboard reorders itself after every vote, that is selection sort running in the background. Connecting these concepts to something as relatable as an election made the learning feel genuinely meaningful rather than just an academic exercise.

The goal was never just to build something that works, it was to build something where you can actually see the data structures doing their job, in a way that makes sense even to someone who has never written a line of code.

1.2 Objectives

The primary objectives of the **Tower of Hanoi DSA Project** are as follows:

- **To simulate a real-world voting process using core data structures** the main goal was to build something that actually mirrors how a voting system works voters lining up, getting verified, casting their vote, and results updating live. Every step of that process is handled by a data structure that genuinely fits the task, not just one chosen arbitrarily to tick a box.
- **To demonstrate that data structures solve real problems, not just exam questions** A lot of the time, data structures feel abstract when you learn them in isolation. This project was an attempt to change that to show that a stack is not just a concept you draw on paper, it is what makes an undo button possible. A hash table is not just theory; it is what prevents someone from voting twice.
- **To build an interactive interface that makes the system visible and understandable** Having the logic work behind the scenes is one thing, but being able to watch it happen seeing the queue fill up, the leaderboard shift, the activity log update makes it far easier to understand what is actually going on. The Qt-based GUI was built with that transparency in mind.
- **To strengthen the connection between classroom theory and practical implementation** This project sits at the intersection of two things: what we learn in a DSA course and what actually goes into building a functional software system. The objective was to bridge that gap to walk away not just with a working application, but with a much clearer understanding of why these data structures were invented in the first place.

1.3 Motivation and Significance

The motivation behind this project arises from the need to make Data Structures and Algorithms concepts more visual, practical, and easier to connect to real-world scenarios. In traditional learning environments, data structures such as Stacks, Queues, and Hash Tables are often taught through simple console-based examples that demonstrate basic operations in isolation. While these examples help build foundational understanding, they rarely show how these structures behave when combined together inside a complete, functioning system. This project addresses that gap by embedding each data structure into a meaningful role within a graphical election application, where users can directly observe how a Queue lines up voters in order, how a Stack enables vote reversal, and how a Hash Table silently prevents duplicate entries — all happening together in real time through an interactive interface.

This project is also significant because it integrates multiple data structures not as separate exercises but as cooperative components of a single working system. The Linked List maintains the candidate registry dynamically, the Queue manages the voter processing pipeline in a First-In-First-Out manner, the Stack preserves vote history to support the undo functionality, the Binary Search Tree validates candidate IDs efficiently during processing, the Hash Table ensures each voter can only cast one vote, and Selection Sort continuously reorders the leaderboard based on live vote counts. This level of integration demonstrates the relationship between theoretical knowledge and practical software design in a way that isolated examples simply cannot.

Furthermore, the use of the Qt Framework provides a professional graphical user interface that makes the entire system transparent and easy to follow. The live leaderboard, activity log, voter queue display, and candidate selection panel give users full visibility into what the system is doing and why. The project not only strengthens programming and problem-solving skills but also serves as an effective educational tool for visualizing how data structures operate in a real application. Its practical implementation, clean visual design, and meaningful context make it well suited for academic evaluation, project demonstration, and viva examinations.

Chapter 2 Related Works

Several research studies and educational tools have explored the development of electronic voting systems and the visualization of data structures and algorithms. Online platforms such as VisuAlgo and Algorithm Visualizer provide interactive, web-based demonstrations of various data structures including Stack, Queue, and Hash Table operations. However, these platforms are designed as general-purpose visualization tools and do not integrate a complete election workflow where multiple data structures work together within a single standalone desktop application.

Electronic voting systems have been widely studied in both academic and government contexts. Existing implementations, such as those explored in research on e-governance and digital democracy, primarily focus on security, cryptography, and network-based voting protocols. While these systems address real-world deployment concerns, they do not emphasize the role of fundamental data structures in managing the voting pipeline, nor do they provide a visual and interactive interface suitable for educational demonstration. Most such systems are also web-based or server-dependent, making them difficult to use as standalone learning tools in a classroom setting.

Traditional academic implementations of voter management and election simulations are commonly taught in Data Structures and Algorithms courses using console-based C++ programs. These implementations mainly focus on demonstrating individual data structure operations such as enqueue, dequeue, push, and pop, without incorporating graphical visualization or a cohesive real-world context. Standard textbooks such as Introduction to Algorithms by Thomas H. Cormen and Data Structures, Algorithms, and Applications in C++ by Sartaj Sahni provide comprehensive theoretical explanations of Stack, Queue, Hash Table, and Binary Search Tree, which form the conceptual foundation for this project. However, they do not provide practical GUI-based implementations that connect these structures within a single interactive application.

In addition, some educational applications have been developed using the Qt Framework to visualize algorithms such as sorting and searching techniques. While these applications demonstrate individual algorithm behavior effectively, implementations that explicitly connect a real-world election process with multiple integrated data structures — including duplicate prevention via Hash Table, candidate validation via BST, ordered processing via Queue, and vote reversal via Stack — along with a live graphical interface, are not widely documented. This project addresses that gap by combining election system design, graphical visualization, and the direct implementation of six fundamental data structures within a single interactive desktop application. Furthermore, the project incorporates a real-time leaderboard, activity logging, and a clean Qt-based interface to enhance usability, making it both an effective educational tool and a functional demonstration of core Data Structures and Algorithms concepts.

Chapter 3 Design and Implementation

3.1 Implementation

The system was implemented using an object-oriented programming approach in C++, with the graphical user interface developed using the Qt Framework. The project is organized into several core components — Candidate, VoterNode, VoteRecord, and MainWindow — each designed with clearly defined roles and responsibilities to ensure modularity, maintainability, and clarity. This structured approach allows a clean separation between data structure implementation, election logic, and the graphical interface, making the codebase easy to follow, extend, and present.

Special emphasis was placed on ensuring that each data structure was not only implemented internally but also reflected through visible system behavior that users can observe in real time. The Candidate structure forms the node of a Singly Linked List that maintains the complete registry of candidates, allowing dynamic traversal and vote updates throughout the session. The VoterNode structure serves as the building block of the Queue, which manages the voter processing pipeline in strict First-In-First-Out order, ensuring fairness in how votes are handled. The VoteRecord structure is used by the Stack to maintain a history of successfully processed votes, enabling the undo functionality that allows the most recent vote to be reversed at any point.

Candidate ID validation is handled internally by a Binary Search Tree, which provides efficient search operations during vote processing without requiring linear traversal of the candidate list. A Hash Table with chaining is used to store voter IDs that have already cast their vote, providing constant-time duplicate detection and preventing any voter from influencing the election more than once. The live leaderboard is generated by applying Selection Sort on the candidate list at every update, ensuring that rankings always reflect the current vote counts accurately.

The MainWindow class serves as the central controller of the application, managing all user interactions, connecting interface events to the underlying data structures, and keeping the display synchronized with the state of the system at all times. The graphical interface built with Qt provides real-time feedback through a candidate selection panel, voter queue display, live leaderboard, and an activity log that records every action taken during the session. This integrated design ensures that the system functions not only as a working election simulation but also as a clear and transparent demonstration of how fundamental data structures operate within a real-world application.

3.1.1 Implementation Planning

The implementation was carefully planned to demonstrate six fundamental DSA concepts: Linked List, Queue, Stack, Binary Search Tree, Hash Table, and Selection Sort, each serving a specific and meaningful role within the election system. The Linked List was implemented to maintain the candidate registry dynamically, allowing candidate records to be stored and traversed sequentially throughout the session. The Queue was implemented to manage the voter processing pipeline following strict First-In-First-Out order, ensuring that every voter is handled fairly and in the sequence they arrived. The Stack was designed to maintain a history of successfully cast votes, enabling the undo functionality that allows the most recent vote to be reversed at any time.

The Binary Search Tree was implemented to handle candidate ID validation efficiently during vote processing, avoiding the need for slow linear searches. The Hash Table was designed with chaining to store processed voter IDs and detect duplicate votes in constant time, forming the core fraud prevention mechanism of the system. Selection Sort was integrated into the leaderboard update cycle, reordering candidates by vote count after every processed vote to ensure results are always accurately ranked.

During the planning phase, Qt Framework was selected as the primary tool for the graphical interface due to its flexibility, cross-platform compatibility, and rich set of UI components. The development process was divided into several key tasks including the implementation of core data structures, election logic, voter queue management, real-time leaderboard updates, activity logging, and the design of an interactive interface that keeps all system behavior visible and easy to follow for both users and evaluators.

3.1.2 Requirement Analysis

A candidate panel must be displayed on the left side of the interface, showing each registered candidate along with their name, party affiliation, current vote count, and a button that allows the user to make their selection. Before any vote can be submitted, the user is required to pick a candidate from the panel and provide a valid voter ID through the input field available in the voting section. The system must validate that a candidate has been selected and a voter ID has been entered before allowing any vote to be queued, displaying appropriate warning messages in the activity log when either condition is not met.

The voter queue must follow strict First-In-First-Out order, processing voters in the exact sequence they joined. The system must prevent duplicate voting by checking each voter ID against the Hash Table before processing, and must reject and log any voter who has already cast a vote. Candidate ID validation must be performed using the Binary Search Tree during processing to ensure only registered candidates receive votes. A process next button must handle one voter at a time, while a process all button must clear the entire queue sequentially. An undo button must reverse the most recently accepted vote using the Stack, restoring the previous vote count and removing the voter ID from the Hash Table.

The live leaderboard must update automatically after every processed vote, reordering candidates using Selection Sort and clearly highlighting the leading candidate. An activity log must record every system event including queued voters, accepted votes, rejected duplicates, and undone votes, each displayed with a distinct color for quick identification. The stat panel at the top must reflect the current total votes cast, number of voters in the queue, and total rejected entries at all times throughout the session.

Non-Functional Requirements

The application must run reliably without crashes, maintain a consistent frame rate of 60 FPS for smooth animations, and provide responsive user input handling. The interface should be clean and intuitive with proper visual feedback for button hover states and active modes. The system must correctly implement all data structure operations and accurately reflect their behavior through the visual representation.

3.1.3 System Design

layered architecture was adopted for this system to ensure a clean separation between election logic and graphical presentation, keeping each component focused on a single responsibility. This structure makes the codebase easier to maintain, test, and present, with each layer communicating in a controlled and predictable direction.

The MainWindow class sits at the top of this architecture and serves as the central controller of the entire application. It is responsible for handling all user interactions including candidate selection, voter ID input, button clicks for joining the queue, processing votes, and triggering the undo operation. Whenever a user action occurs, the MainWindow communicates downward to the underlying data structures to perform the required operation and then updates all visible interface components to reflect the new system state.

The data structure layer sits beneath the interface and handles all core election logic independently of the graphical presentation. The Candidate List manages the linked list of registered candidates and provides methods for retrieval and vote updates. The Voter Queue manages the processing pipeline and enforces First-In-First-Out order at all times. The Vote Stack maintains the history of accepted votes to support undo functionality. The Candidates performs candidate ID validation during processing, and the VoterHashTable tracks voted IDs to prevent duplicate entries. None of these components directly modify the interface, ensuring that logic and presentation remain fully separated.

Communication between layers is strictly one-directional. The MainWindow calls methods in the data structure layer to trigger operations, while the data structure layer returns results without ever reaching back into the interface. This separation ensures modularity, maintainability, and clarity throughout the system, making it straightforward to follow the flow of any operation from the moment a user interacts with the interface to the point where the result is reflected back on screen.

Stack Implementation

The Stack in this project is implemented manually using a custom singly linked node structure, without relying on any standard library stack container. Two supporting structures form the foundation of this implementation. The VoteRecord structure stores the details of a single processed vote, holding the voter ID and the corresponding candidate ID. The Stack Node structure wraps a VoteRecord and contains the next node

in the chain, allowing the stack to grow and shrink dynamically through pointer manipulation alone.

The Vote Stack class manages these nodes through a single top pointer that always references the most recently added entry. When a vote is successfully accepted and processed, the push operation creates a new Stack Node containing the vote details, points it to the current top, and updates the top pointer to the new node. This ensures that the most recent vote is always accessible at the top of the stack following the Last-In-First-Out principle. When the user triggers the undo functionality, the pop operation retrieves the VoteRecord from the top node, moves the top pointer down to the next node, and deletes the removed node to free memory. The isEmpty check prevents any undo attempt when no votes have been recorded, displaying an appropriate warning in the activity log instead.

In terms of system behavior, every successful vote processed through the queue is immediately pushed onto the stack. When undo is triggered through the interface, the most recent vote is popped from the stack, the corresponding candidate's vote count is decremented, and the voter ID is removed from the Hash Table — completely reversing the vote as though it was never cast. This makes the Stack an essential component of the system's reliability, giving users the ability to correct mistakes without restarting the entire session.

Queue Implementation

The Queue in this project is implemented manually using a custom singly linked node structure, following strict First-In-First-Out order to manage the voter processing pipeline. The VoterNode structure holds the voter ID and the selected candidate ID, along with a pointer to the next node in the chain. The VoterQueue class manages these nodes through two pointers — front and rear — where front always points to the next voter to be processed and rear points to the most recently added entry. When a voter joins the queue through the interface, the enqueue operation creates a new VoterNode and attaches it to the rear of the chain. When a vote is ready to be processed, the dequeue operation removes the node at the front, retrieves its details for validation and processing, and moves the front pointer forward. This ensures that voters are always handled in the exact order they joined, accurately reflecting a real-world election queuing model. The queue also provides

a snapshot method that returns the current list of waiting voters, which is used by the interface to keep the voter queue display updated after every operation.

Recursion Tree

The BST in this system contains four nodes corresponding to the four registered candidate IDs, giving it a depth of three and a total of four nodes in its structure. Search operations on this tree run at an average time complexity of O(log n), making it significantly faster than a linear search through the linked list as the number of candidates grows. If an invalid or unregistered candidate ID is somehow submitted for processing, the BST search returns false and the vote is immediately rejected and logged as an error in the activity log. This makes the Binary Search Tree the first line of defense in ensuring that only legitimate candidates receive votes, contributing directly to the integrity and reliability of the election system.

Undo Stack

Every successfully processed vote is immediately pushed onto the VoteStack, which contains a VoteRecord struct storing the voter ID and the candidate ID that received the vote. When the Undo button is pressed by the user, the most recent entry is popped from the top of the stack and the reversal process begins the corresponding candidate's vote count is decremented by one, and the voter ID is removed from the Hash Table so that the same voter is no longer marked as having voted. This provides an O(1) undo operation following strict Last-In-First-Out ordering, ensuring that the most recently accepted vote is always the one reversed first. The undo feature adds an important layer of reliability to the system, allowing operators to immediately correct any mistakenly processed vote without restarting the session or losing any other recorded data. It also reinforces the practical value of the Stack as a data structure, demonstrating how LIFO behavior maps naturally onto the concept of reversing the most recent action in a real-world application.

GUI Interaction and Animation

The graphical interface is built using the Qt Framework, providing a clean and interactive desktop environment where all election operations are visible and accessible from a single window. The interface is divided into two main columns the left side handles candidate selection, voter ID input, action buttons, and the voter queue display, while the right side presents the live leaderboard and the activity log. Users interact with the system through a straightforward workflow where they first click on a candidate card to make their selection, enter a voter ID into the input field, and then press the Join Queue button or hit Enter to add the voter to the processing pipeline. The process next and process all buttons trigger vote processing, while the undo button reverses the last accepted vote instantly. Every action taken by the user produces immediate visual feedback across all interface components — the queue display updates to show the current waiting voters, the leaderboard reorders itself to reflect the latest vote counts, the stat panel at the top adjusts the figures for total votes cast, voters in queue, and rejected entries, and the activity log appends a new color-coded entry describing exactly what happened. This real time synchronization between user interaction and interface response ensures that the system always reflects an accurate and up to date picture of the election state, making it both easy to operate and straightforward to demonstrate during academic evaluation.

Educational Features

An integrated About/DSA Info dialog provides visual explanations of the Stack, Queue, and Recursion Tree implementations. It includes code snippets, diagrams of the recursion tree, and interactive examples of tower states. This feature bridges the gap between theoretical DSA concepts and practical implementation, making the project both a game and a learning tool. The GUI also displays the current move count and game timer, reinforcing concepts of algorithm efficiency and sequence tracking.

3.1.4 Development

The development of this project was carried out in stages to ensure that each component was correctly implemented and stable before moving on to the next, resulting in a system that is both modular and reliable. The first phase focused entirely on designing and implementing the core data structures in isolation. Each structure — the Candidate Linked List, Voter Queue, Vote Stack, Candidate BST, Voter Hash Table, and Selection Sort — was written and tested independently to confirm that insertion, deletion, search, and traversal operations behaved correctly before any graphical layer was introduced. This phase established a solid and verified foundation for all election logic that the rest of the system would depend on.

The second phase involved building the Qt graphical interface around the already validated core logic. The main window layout was designed with careful attention to how information would be presented, ensuring that the candidate panel, voter queue display, leaderboard, stat panel, and activity log were all visible and logically arranged within a single window. Each interface component was then connected to its corresponding data structure through Qt signal and slot connections, so that every user action would trigger the correct operation and immediately reflect its result across all visible elements of the interface.

The third phase focused on integration and real time synchronization between the interface and the underlying data structures. A key challenge during this phase was ensuring that all interface components updated consistently after every operation, particularly the leaderboard which required Selection Sort to run and all candidate cards to rebuild correctly after each vote. This was resolved by implementing a central refresh All method in the MainWindow class that rebuilds and synchronizes every interface component in the correct order after any state change, ensuring that no display ever shows outdated or inconsistent information.

The major development phases of the project can be summarized as follows:

1. **Data structure design and implementation** — Linked List, Queue, Stack, BST, Hash Table, and Selection Sort written and tested independently.
2. **Core election logic** — Voter enqueue, vote processing, duplicate detection, candidate validation, and undo functionality connected and verified.
3. **Qt GUI layout** — Main window, candidate panel, leaderboard, queue display, stat panel, and activity log designed and arranged.
4. **Signal and slot integration** — All buttons, input fields, and interface components connected to their corresponding data structure operations.
5. **Synchronization and refresh system** — Central refresh All method implemented to keep all interface components consistently updated after every operation.

3.1.5 Testing and Debugging

The project underwent thorough testing across all major functionalities to verify correctness, stability, and consistent behavior under different usage scenarios. Testing was carried out iteratively throughout the development process, with each data structure validated individually before being tested as part of the integrated system. Several issues were identified and resolved during this process, ranging from logical errors in data structure behavior to interface synchronization problems.

The first major issue encountered was related to the Hash Table's duplicate detection mechanism. During early testing, it was observed that voter IDs entered with different letter casing — for example V-001 and v-001 — were being treated as two separate entries, allowing the same voter to cast more than one vote. This was resolved by converting all voter IDs to uppercase using Qt's toUpper() method before any enqueue or lookup operation, ensuring that the Hash Table always receives a consistently formatted key regardless of how the user typed the input.

The second issue involved the undo functionality and its interaction with the Hash Table. When a vote was undone using the Stack, the candidate's vote count was correctly decremented, but the voter ID was not always being removed from the Hash Table in certain edge cases, which meant the same voter could not vote again after an undo even though their vote had been reversed. This was resolved by ensuring that the remove method of the VoterHashTable was always called immediately after the pop operation in the undo Last slot, keeping the Stack and Hash Table in perfect synchronization at all times.

The third issue was a display inconsistency in the leaderboard. After processing all voters using the process all button, the leaderboard was occasionally showing candidates in the wrong order for a single frame before correcting itself. This was caused by the refresh All method rebuilding the candidate cards before the leaderboard, resulting in a brief moment where vote counts were updated but sorting had not yet been applied. The issue was resolved by reordering the internal calls within refresh All so that Selection Sort always runs and the leaderboard is fully rebuilt before the candidate panel is updated.

In addition to these primary issues, several smaller bugs were identified and addressed during testing. Entering an empty voter ID and clicking join queue produced no warning in early builds, which was fixed by adding an explicit isEmpty check before any queue operation. Processing a vote when no candidate was selected caused the system to reference a null candidate pointer, which was resolved by validating the selectedCandidateId against negative one before allowing any queue entry. The stat panel also failed to update the rejected count immediately after a duplicate was detected, which was corrected by ensuring update Stats was always called at the end of every process Next operation regardless of whether the vote was accepted or rejected.

Through iterative testing and careful debugging, the system now performs reliably across all operations, providing a stable, accurate, and visually consistent experience that correctly demonstrates the behavior of all six integrated data structures throughout a complete election session.

3.2 System Requirement Specifications

The system was developed and tested with specific software and hardware requirements to ensure proper functionality, performance, and compatibility. The specifications are detailed below.

3.2.1 Software Specifications

The following software components are required for the development and execution of the system:

- **Programming Language:** C++17
- **GUI Framework:** Qt 5.15 or Qt 6.x (Qt Widgets module)
- **Integrated Development Environment (IDE):** Qt Creator
- **Compiler:** MinGW (Windows) or GCC (Linux/Mac)
- **Build System:** qmake (.pro project file)

These components provide a stable foundation for implementing both the backend logic and the graphical frontend, enabling efficient development, debugging, and testing of the Tower of Hanoi application.

3.2.2 Hardware Specifications

The system requires the following hardware to operate efficiently:

- **Processor:** Intel Core i3 or equivalent (minimum); Intel Core i5 or higher recommended
- **Memory (RAM):** 4 GB minimum; 8 GB recommended
- **Storage:** 500 MB minimum for Qt installation and project files
- **Display:** 1366×768 resolution minimum; 1920×1080 recommended
- **Input Devices:** Standard keyboard and mouse

These specifications ensure smooth execution of the system, support real-time animations, and provide a responsive and user-friendly experience while interacting with the game and visualizing data structure operations.

Chapter 4 Discussion on the achievements

4.1 Project Overview

The Nepal E-Election Vote Tallying System is a C++ desktop application developed using the Qt Framework, presenting a complete simulation of an electronic voting process while explicitly demonstrating six core data structures: Linked List, Queue, Stack, Binary Search Tree, Hash Table, and Selection Sort. The application features a dark themed graphical interface divided into two main columns, with the left side handling candidate selection, voter input, action buttons, and the voter queue display, and the right side presenting the live leaderboard and a real time activity log. The system comes preloaded with four registered candidates — Ram Bahadur of Nepali Congress, K.P. Sharma Oli of CPN-UML, Balendra Shah of RSP Nepal, and Amrit Rasaili as an Independent — each displayed as an interactive card showing their name, party, current vote count, and vote percentage.

The top section of the interface displays three live stat cards showing the total votes cast, the number of voters currently waiting in the queue, and the total number of rejected entries recorded during the session. The voting workflow follows a straightforward sequence — the user selects a candidate, enters a voter ID, and joins the queue, after which votes can be processed one at a time or all at once. Four action buttons drive the core functionality: Join Queue, Process Next, Process All, and Undo. The activity log on the right records every system event with color coded entries distinguishing between accepted votes, rejected duplicates, queued voters, and undone votes, providing full transparency into every operation performed throughout the session.

4.2 Implemented Features

4.2.1 DSA Implementations

Stack

The Stack in this project is implemented manually using a custom singly linked node structure, without relying on any standard library stack container. Two supporting structures form the foundation of this implementation. The VoteRecord structure stores the details of a single processed vote, holding the voter ID and the corresponding candidate ID. The Stack Node structure wraps a VoteRecord and contains the next node in the chain, allowing the stack to grow and shrink dynamically through pointer manipulation alone.

The Vote Stack class manages these nodes through a single top pointer that always references the most recently added entry. When a vote is successfully accepted and processed, the push operation creates a new Stack Node containing the vote details, points it to the current top, and updates the top pointer to the new node. This ensures that the most recent vote is always accessible at the top of the stack following the Last-In-First-Out principle. When the user triggers the undo functionality, the pop operation retrieves the VoteRecord from the top node, moves the top pointer down to the next node, and deletes the removed node to free memory. The isEmpty check prevents any undo attempt when no votes have been recorded, displaying an appropriate warning in the activity log instead.

Queue

The Queue in this project is implemented manually using a custom singly linked node structure, following strict First-In-First-Out order to manage the voter processing pipeline. The VoterNode structure holds the voter ID and the selected candidate ID, along with a pointer to the next node in the chain. The VoterQueue class manages these nodes through two pointers — front and rear — where front always points to the next voter to be processed and rear points to the most recently added entry. When a voter joins the queue through the interface, the enqueue operation creates a new VoterNode and attaches it to the rear of the chain. When a vote is ready to be processed, the dequeue operation removes the node at the front, retrieves its details for validation and processing, and moves the front pointer forward. This ensures that voters are always handled in the exact order they joined, accurately reflecting a real-world election queuing model.

Undo Stack

Every successfully processed vote is immediately pushed onto the VoteStack, which contains a VoteRecord struct storing the voter ID and the candidate ID that received the vote. When the Undo button is pressed by the user, the most recent entry is popped from the top of the stack and the reversal process begins the corresponding candidate's vote count is decremented by one, and the voter ID is removed from the Hash Table so that the same voter is no longer marked as having voted. This provides an O(1) undo operation following strict Last-In-First-Out ordering, ensuring that the most recently accepted vote is always the one reversed first. The undo feature adds an important layer of reliability to the system, allowing operators to immediately correct any mistakenly processed vote without restarting the session or losing any other recorded data. It also reinforces the practical value of the Stack

as a data structure, demonstrating how LIFO behavior maps naturally onto the concept of reversing the most recent action in a real-world application.

Recursion Tree

The BST in this system contains four nodes corresponding to the four registered candidate IDs, giving it a depth of three and a total of four nodes in its structure. Search operations on this tree run at an average time complexity of O(log n), making it significantly faster than a linear search through the linked list as the number of candidates grows. If an invalid or unregistered candidate ID is somehow submitted for processing, the BST search returns false and the vote is immediately rejected and logged as an error in the activity log. This makes the Binary Search Tree the first line of defense in ensuring that only legitimate candidates receive votes, contributing directly to the integrity and reliability of the election system.

4.2.2 GUI and Interaction

When you first open the application, the first thing you notice is the dark themed interface that feels clean and professional. Everything you need is visible on a single screen without any need to navigate between pages or open separate windows. The layout is divided into two sides that work together — the left side is where all the action happens, and the right side is where you watch the results unfold in real time.

At the very top of the window sits a header section that immediately tells you what this system is about. On the left of the header, you can see the title of the system along with three small badge tags that read LIVE, COMP 202 DSA, and KATHMANDU UNIVERSITY. Below the title there is a subtle line listing all six data structures used in the project. On the right side of the header are three stat cards that display the total votes cast, the number of voters currently sitting in the queue, and the total number of rejected entries. These numbers are not static — they update themselves automatically every single time something happens in the system, so you always know the current state of the election at a glance without having to look anywhere else.

Moving down to the left column, the first thing you see is the candidate panel. There are four candidate cards stacked vertically, each one showing the candidate's avatar with their initial, their full name, their party affiliation, a thin progress bar showing their share of the total votes, their current vote count in large text, and their vote percentage right below it. On the right side of each card there is a select button. When you click that button the card changes its appearance — the background gets a subtle tint of the candidate's color and the button text changes to show a checkmark and the word Selected, giving you a clear visual confirmation that your choice has been registered. Only one candidate can be selected at a time, and clicking a different card immediately switches the selection over.

Just below the candidate panel is the vote casting section, wrapped in a slightly lighter card to separate it visually from the rest. This is where you type in the voter ID. The input field has a dark background with a monospace font and a placeholder text that gives you an example of the expected format. When you click on the field it gets a colored border highlight so you know it is active. Below the input field are four buttons arranged in a single row. The Join Queue button is the most prominent one — it is bright red and takes up the most space, making it obvious that this is the primary action. Next to it are the Process Next, Process All, and Undo buttons, which are styled more subtly with outline text in cyan, yellow, and pink respectively so they feel secondary without being hard to find. You can also just press Enter after typing the voter ID instead of clicking Join Queue, which makes the workflow feel faster when processing multiple voters.

Below the vote casting section is the voter queue display. This is a dark scrollable list that shows every voter currently waiting to be processed. Each entry shows the voter ID, an arrow, the candidate name they voted for, and the candidate ID in brackets. When the queue is empty the list shows a quiet message saying the queue is empty in a muted color so it does not feel alarming. As voters join the queue the list fills up from top to bottom, and as they get processed the entries disappear from the top one by one, visually demonstrating the First-In-First-Out behavior of the queue in a way that is immediately understandable.

On the right side of the window the first thing you see is the live leaderboard. This section shows all four candidates ranked in order of their current vote count, and it re-sorts itself after every single vote is processed. The candidate in first place gets a gold-colored rank badge and a subtle golden background tint on their card, making the winner visually obvious even from a distance. Second and third place get silver and bronze styling respectively. Each leaderboard card shows the rank, a small colored dot matching the candidate's color, the candidate’s name, their party, a small progress bar, the vote count in large text, and their percentage. When a candidate takes the lead for the first time their name gets a star symbol next to it, adding a small but satisfying visual moment to the experience.

Below the leaderboard is the activity log, which is arguably the most informative part of the entire interface. Every single thing that happens in the system gets recorded here as a new line entry. Accepted votes appear in green, rejected duplicates appear in red, queued voters appear in cyan, and undone votes appear in pink. Each entry has a slightly tinted background matching its color so you can scan through the log quickly and immediately understand what happened and when. The log scrolls automatically to the latest entry so you never have to scroll down manually. Over the course of a full voting session the log builds up into a complete record of everything that took place, which makes it very useful for demonstrating the system during a presentation or viva because you can point to specific entries and explain exactly what data structure was responsible for that particular outcome.

The overall interaction flow feels natural and deliberate. You select a candidate, type a voter ID, hit Enter or click Join Queue, watch the queue update, click Process Next to handle one voter at a time or Process All to clear the entire queue at once, and watch the leaderboard shift and the log fill up with results. If something goes wrong or a mistake is made you hit Undo and the last vote disappears cleanly. The whole experience is designed so that every action has an immediate and visible consequence somewhere on the screen, making it easy to follow what is happening and why.

4.3 Achievements

During the development of the Nepal E-Election Vote Tallying System, a wide range of technical and problem-solving skills were acquired and strengthened throughout the process. A complete object-oriented design was planned and implemented from scratch, comprising the Candidate, VoterNode, VoteRecord, and MainWindow components, with a clear and deliberate separation of responsibilities between data structure logic, election processing, and the graphical interface. The project explicitly demonstrates the practical use of six fundamental data structures — Linked List, Queue, Stack, Binary Search Tree, Hash Table, and Selection Sort — all integrated into a single coherent application where each structure plays a distinct and meaningful role rather than existing in isolation.

Working through the challenges of connecting multiple data structures within a live graphical system developed a much deeper understanding of how these structures behave not just in theory but under real operating conditions. Resolving issues such as the Hash Table's case sensitivity bug, the undo and Hash Table synchronization problem, and the leaderboard ordering inconsistency required careful analysis and methodical debugging, which significantly improved problem-solving and critical thinking skills. Each bug encountered was not just a technical obstacle but an opportunity to better understand the relationship between data structure behavior and interface state management.

Implementing the real time synchronization system through the central refreshAll method provided valuable experience in managing application state across multiple interdependent interface components, ensuring that the candidate panel, leaderboard, queue display, stat cards, and activity log always reflected a consistent and accurate picture of the system at any given moment. This reinforced a practical understanding of Qt's signal and slot architecture and how user interaction events can be cleanly connected to underlying logic without creating tight coupling between layers.

Overall, this project transformed the theoretical knowledge gained in the Data Structures and Algorithms course into a tangible, working, and visually demonstrable system. It strengthened skills in C++ programming, object-oriented design, Qt GUI development, memory management, and real world application of core data structures, making it a comprehensive and rewarding learning experience that goes well beyond what any console-based assignment could offer.

Chapter 5 Conclusion and Recommendation

The Nepal E-Election Vote Tallying System successfully demonstrates that Data Structures and Algorithms are not just academic concepts confined to textbooks and lab exercises — they are practical tools that solve real problems in real systems. Every feature in this application exists because a data structure made it possible. The Queue keeps voting fair. The Hash Table keeps it honest. The Stack makes it forgiving. The BST keeps it fast. The Linked List keeps it organized. And Selection Sort keeps the results clear.

Building this project made the theory click in a way that no console program ever could. Watching a duplicate voter get rejected in real time, seeing the leaderboard shift after every vote, and hitting undo to instantly reverse a mistake — these moments turned abstract concepts into something that could actually be seen, used, and understood.

At its core this project is proof that when data structures are given the right context, they stop being something you study for an exam and start being something you genuinely understand.

5.1 Limitations

The limitations are as follows:

1. Fixed Number of Candidates The system currently supports only four hardcoded candidates that are registered at startup. There is no functionality to add, remove, or edit candidates during runtime, which means the system cannot be adapted to a real election scenario where the number of candidates may vary depending on the constituency or election type.

2. No Data Persistence All election data including votes cast, voter IDs, queue entries, and results exist only for the duration of the application session. Once the application is closed everything is lost permanently. There is no database or file system integration to save and reload election records, making it impossible to resume a session or review past results.

3. Single Session Voting Only The system is designed to handle one election session at a time with no support for multiple simultaneous voting booths or

districts. In a real-world election, votes would be collected from many locations and merged together, which is something the current architecture does not support.

4. No Authentication or Security Layer Voter IDs are entered manually by the operator with no verification against any registered voter database. Anyone can type any ID and join the queue, meaning the system relies entirely on the honor system for input accuracy. There is no login, encryption, or access control of any kind protecting the election process.

5. Undo is Limited to One Vote at a Time The undo functionality can only reverse votes one by one in reverse order, and it cannot selectively undo a specific vote from the middle of the history. If ten votes have been processed and only the fifth one needs to be corrected, the operator would have to undo all ten votes one at a time and reprocess them, which is impractical in a real election environment.

5.2 Future Enhancement

1. Dynamic Candidate Registration A future version of the system could include a candidate management panel that allows administrators to add, edit, or remove candidates before the election session begins. This would make the system far more flexible and adaptable to different election scenarios, removing the current limitation of having candidates hardcoded at startup. Each newly registered candidate would automatically be inserted into the Linked List and BST without requiring any changes to the source code.

2. Database Integration for Data Persistence Integrating a lightweight database such as SQLite would allow the system to save all election data permanently, including voter records, vote counts, rejected entries, and session logs. This would make it possible to close and reopen the application without losing any data, generate post-election reports, and maintain a complete and auditable history of every action taken during the session, bringing the system much closer to a production-ready election tool.

3. Voter Authentication and Registration System A proper voter registration module could be added where voters are pre-registered with verified IDs before the election begins. During voting, the system would cross-check the entered ID against the registered voter database before allowing anyone to join the queue. This would eliminate the current reliance on manual input accuracy and add a genuine layer of security and legitimacy to the voting process.

4. Multi-Constituency and Multi-Session Support The system could be extended to support multiple constituencies or voting districts running simultaneously, each with their own candidate list, voter queue, and results. A central results panel could aggregate votes from all active sessions and display a combined national leaderboard in real time. This would significantly increase the scalability of the system and make it a much more realistic simulation of how a large-scale election actually operates.

References

1. Ganesh-1433 (2025). Secure online voting system using c programming.
2. Uttamshr10 (2025). Election management system using java swing.
3. Wikipedia contributors (2026). Election.

APPENDIX

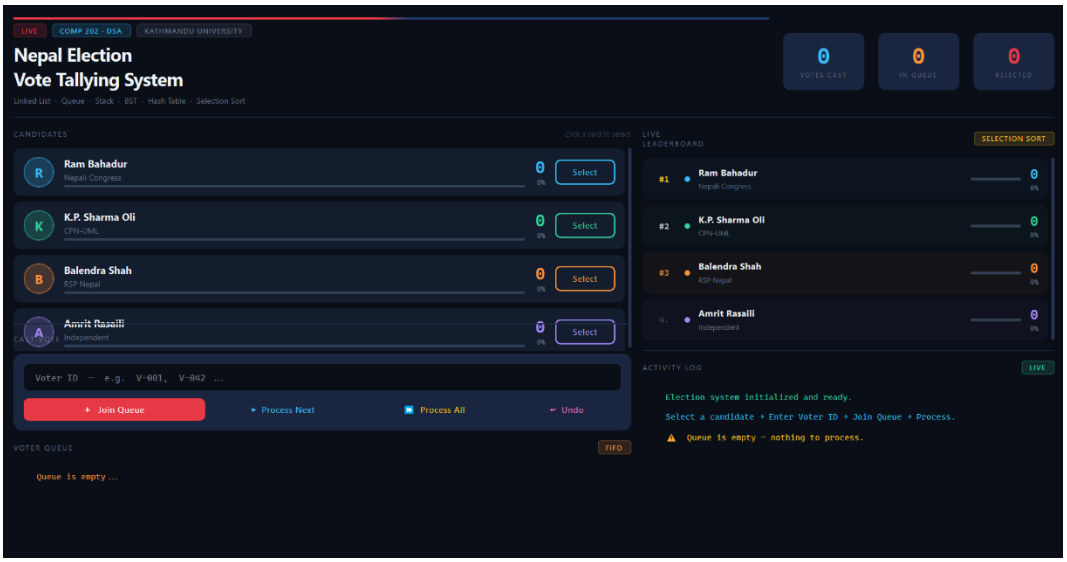

Fig: e-election